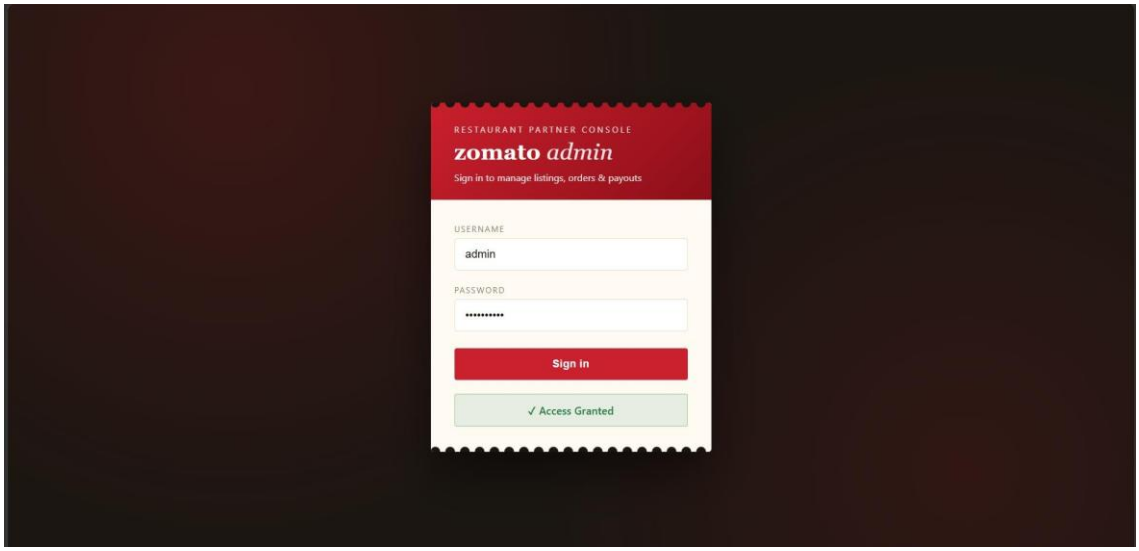
task

1) Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

```
    <!DOCTYPE html>
<html lang="en">
    <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mock Zomato Admin Login</title>
    </head>
    <body>

    <div class="login-box">
    <h2>Zomato Admin Login</h2>

    <input type="text" id="username" placeholder="Username">
    <input type="password" id="password"
    placeholder="Password">
    <button onclick="login()">Login</button>

<p id="message"></p>
    </div>

    <script>
    // Hardcoded mock credentials
    const ADMIN_USERNAME = "admin";
    const ADMIN_PASSWORD =
    "zomato123";
    function login() {         const username =
    document.getElementById("username").value;         const
    password = document.getElementById("password").value;
    const message = document.getElementById("message");
    if (username === ADMIN_USERNAME && password === ADMIN_PASSWORD) {
    message.textContent = "Access Granted";
    message.style.color = "green";
} else {
    message.textContent = "Invalid username or password";
    message.style.color = "red";
    }
}
    </script>

    </body>
    </html>
```



2) Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.

Hint: Use a dictionary to map roles to allowed resources.

```
def check_access(user_role, resource):
  """
   Check if a user role has access to a given resource.    Returns True if
allowed, False otherwise.
  """
access_map = {
     'customer': {'order_history'},
     'restaurant_owner': {'menu_edit', 'order_history'},
     'admin': {'menu_edit', 'order_history', 'admin_dashboard'},
  }    allowed_resources = access_map.get(user_role, set())
return resource in allowed_resources

test_cases = [
  ('customer', 'menu_edit'),
  ('customer', 'order_history'),       ('customer',
'admin_dashboard'),
  ('restaurant_owner', 'order_history'),
  ('restaurant_owner', 'admin_dashboard'),
 ('admin', 'admin_dashboard'),
  ('unknown_role', 'menu_edit'),
] for role, resource in test_cases:
  result = check_access(role, resource)    print(f"check_access('{role}',
'{resource}') -> {result}")
```

Output:

```
[Running] python -u "f:\Tops\one.py" check_access('customer',
'menu_edit') -> False check_access('customer',
'order_history') -> True check_access('customer',
'admin_dashboard') -> False check_access('restaurant_owner',
'menu_edit') -> True check_access('restaurant_owner',
'order_history') -> True check_access('restaurant_owner',
'admin_dashboard') -> False check_access('admin',
'admin_dashboard') -> True
check_access('unknown_role', 'menu_edit') -> False
```

3) Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

```
import random

# --- Simulated stored credentials ---
CORRECT_USERNAME = "user123"
CORRECT_PASSWORD = "pass123"


def generate_otp():
    """Generate a random 6-digit OTP."""
return str(random.randint(100000, 999999))


def login():
    print("=== Login (Step 1: Password) ===")
username = input("Enter username: ").strip()
password = input("Enter password: ").strip()
     if username != CORRECT_USERNAME or password !=
CORRECT_PASSWORD:
        print("Invalid username or password.")
return
     print("\nPassword verified
successfully!\n")

    # --- Step 2: OTP generation and verification ---     otp = generate_otp()
print("=== Two-Factor Authentication (Step 2: OTP) ===")     print(f"[SYSTEM]
Your OTP is: {otp}")  # Simulates sending OTP via SMS/email
     entered_otp = input("Enter the 6-digit OTP sent to your device: ").strip()
     if entered_otp ==
otp:
        print("\nLogin Successful")
else:
        print("\nLogin Failed: Incorrect OTP.")


if __name__ == "__main__":
    login()
```

```
=== Login (Step 1: Password) ===
Enter username: user123
Enter password: pass123

Password verified successfully!

=== Two-Factor Authentication (Step 2: OTP) ===
[SYSTEM] Your OTP is: 494921
Enter the 6-digit OTP sent to your device: 494921

Login Successful
```

4) Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.

```
function getActivePremiumUsers(users) {    const
today = new Date("2024-06-10");
   return users.filter(user => {       const lastLogin = new Date(user.lastLogin);
const diffDays = (today - lastLogin) / (1000 * 60 * 60 * 24);
      return user.isPremium && diffDays >= 0 && diffDays <= 7;
  });
}

// Example usage const
users = [
  { username: "Alice", isPremium: true, lastLogin: "2024-06-09" },
  { username: "Bob", isPremium: false, lastLogin: "2024-06-08" },
  { username: "Charlie", isPremium: true, lastLogin: "2024-06-01" },
  { username: "David", isPremium: true, lastLogin: "2024-06-05" },
  { username: "Eve", isPremium: true, lastLogin: "2024-05-30" }
];
console.log(getActivePremiumUsers(users));
```

```
name: "Alice", isPremium: true, lastLogin: "2024-06-09" },
name: "David", isPremium: true, lastLogin: "2024-06-05" }
```

5) Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Access Control Mistake	One-Line Fix / Prevention Tip
1. Trusting only the frontend for permissions	Always validate user permissions on the **server**, not just in the UI.
2. Insecure Direct Object Reference (IDOR) (e.g., changing an order ID in the URL to view another user's data)	Verify that the requested resource belongs to the authenticated user before returning it.
3. Missing role-based access checks (e.g., customers accessing admin pages)	Implement **Role-Based Access Control (RBAC)** and check roles on every protected request.
4. Using predictable resource IDs without authorization checks	Don't rely on hidden or sequential IDs; always perform authorization checks before granting access.
5. Not expiring sessions or authentication tokens	Set appropriate session/token expiration times and invalidate them on logout or password changes.